

CSB24084(Assignment 13)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char *data;
    size_t length;
    size_t capacity;
} StringBuffer;

StringBuffer *sb_init(size_t initial_capacity)
{
    if (initial_capacity == 0)
        initial_capacity = 1;

    StringBuffer *sb = malloc(sizeof(StringBuffer));

    if (!sb) {
        fprintf(stderr, "Failed to allocate StringBuffer\n");
        return NULL;
    }

    sb->data = malloc(initial_capacity);

    if (!sb->data) {
        fprintf(stderr, "Failed to allocate buffer\n");
        free(sb);
        return NULL;
    }

    sb->data[0] = '\0';
    sb->length = 0;
    sb->capacity = initial_capacity;

    return sb;
}

int sb_append(StringBuffer *sb, const char *str)
{
    if (!sb || !str)
        return -1;

    size_t str_len = strlen(str);
    size_t needed = sb->length + str_len + 1;

```

```

if (needed > sb->capacity) {

    size_t new_capacity = sb->capacity;

    while (new_capacity < needed)
        new_capacity *= 2;

    char *temp = realloc(sb->data, new_capacity);

    if (!temp) {
        fprintf(stderr, "realloc failed\n");
        return -1;
    }

    printf(" [growth] capacity %4zu -> %4zu bytes\n",
        sb->capacity, new_capacity);

    sb->data = temp;
    sb->capacity = new_capacity;
}

memcpy(sb->data + sb->length, str, str_len + 1);

sb->length += str_len;

return 0;
}

void sb_free(StringBuffer *sb)
{
    if (!sb)
        return;

    free(sb->data);
    free(sb);
}

void sb_print(const StringBuffer *sb, const char *label)
{
    printf("%-20s | len=%-4zu cap=%-4zu | \"%s\"\n",
        label,
        sb->length,
        sb->capacity,

```

```

        sb->data);
    }

int main()
{
    printf("=====\n");
    printf("    Dynamic String Buffer - Demo\n");
    printf("=====\n\n");

    printf("-- Initialise (capacity = 8) ----- \n");

    StringBuffer *sb = sb_init(8);

    if (!sb)
        return 1;

    sb_print(sb, "after init");

    printf("\n-- Append \"Hello\" ----- \n");

    if (sb_append(sb, "Hello") != 0) {
        sb_free(sb);
        return 1;
    }

    sb_print(sb, "after \"Hello\"");

    printf("\n-- Append \", World\" (triggers growth #1) ----- \n");

    if (sb_append(sb, ", World") != 0) {
        sb_free(sb);
        return 1;
    }

    sb_print(sb, "after \", World\"");

    printf("\n-- Append \"! Have a nice day.\" (triggers growth #2)\n");

    if (sb_append(sb, "! Have a nice day.") != 0) {
        sb_free(sb);
        return 1;
    }

    sb_print(sb, "after long append");
}

```

```

printf("\n-- Append \" :)\n\" (triggers growth #3) -----\\n");

if (sb_append(sb, " :)") != 0) {
    sb_free(sb);
    return 1;
}

sb_print(sb, "after \" :)\n");

printf("\n-- Final buffer -----\\n");

printf(" Content : %s\\n", sb->data);
printf(" Length : %zu characters\\n", sb->length);
printf(" Capacity : %zu bytes\\n", sb->capacity);

printf("\n-- Freeing all memory -----\\n");

sb_free(sb);

printf(" StringBuffer freed successfully.\\n");

return 0;
}

```